Qamel / Timing-Multiuser-Protocols: A Complete Codebase Reference for Collaborating Models

Internal technical memo
(generated for the Claude ↔ Claude Code collaboration loop)

June 19, 2026

Abstract

This document is a self-contained reference to the `Timing-Multiuser-Protocols` repository (project codename *Qamel*). It is written so that another model with a stronger grasp of the physics — entanglement distribution in quantum repeater chains, multipartite generation with fusion, and the latency budget of quantum SDN — can read it once and then meaningfully collaborate on the existing reinforcement learning (RL) implementation. The document does *not* re-derive any code beyond what is necessary to understand the design choices and pitfalls. Where the code disagrees with the README or with the paper's framing, that disagreement is flagged explicitly. The aim is to maximize agent–agent collaboration efficiency: after reading this, the second Claude should be able to suggest physics-aware modifications without further code spelunking.

Contents

1 Latest Implementation and Result Log (through June 19, 2026) 4
- 1.1 Lab notebook entry: Myriad status on June 17, 2026 . . . 4
- 1.2 Lab notebook entry: $n = 6$ no-go and $n = 7$ control status on June 19, 2026 . . . 5
- 1.3 Lab notebook entry: Myriad workflow on June 17, 2026 . . . 8

2 Stage-0 Result Update (June 15, 2026) 11
- 2.1 Stage-0 method in one pass . . . 11
- 2.2 Stage-0 physics in one pass . . . 12
- 2.3 Stage-0 numerical result . . . 12

3 Project Overview and Scope 15
- 3.1 What the repository *actually implements* . . . 15
- 3.2 What the analytical side implements (different problem!) . . . 16
- 3.3 Why the two scopes diverge . . . 16

4 Physical Model 16
- 4.1 Chain dynamics implemented in `RepeaterChain` . . . 16
- 4.2 What is intentionally *not* modelled . . . 17

5 MDP Formulation **18**
5.1 State $s_t \in \mathbb{R}^{3\times n\times n}$ . . . 18
5.2 Action $a_t \in \{0,1\}^{n\times n}$ . . . 18
5.3 Transition $P(s_{t+1} \mid s_t, a_t)$ . . . 19
5.4 Terminal conditions . . . 20
5.5 Reward . . . 20
5.6 Valid-action mask . . . 20

6 The Reinforcement Learning Code **21**
6.1 Tabular Q-learning (baseline) . . . 21
6.2 DQN: observation modes . . . 21
6.3 DQN: network . . . 21
6.4 DQN: training loop . . . 22
6.5 DQN hyperparameters (canonical) . . . 23
6.6 Curriculum learning . . . 23
6.7 Best-eval checkpointing . . . 23
6.8 Resume semantics . . . 24

7 Evaluation Pipeline **24**
7.1 Policy execution . . . 24
7.2 Ablation filters . . . 24
7.3 Tracing . . . 25
7.4 Artifacts . . . 25
7.5 Head-to-head policy comparison . . . 25

8 Multi-Budget Study Pipeline **25**
8.1 Inputs (environment variables) . . . 26
8.2 Loop structure . . . 26
8.3 Cross-budget summary . . . 26

9 Analytical and Monte-Carlo Baseline **26**
9.1 `generate_tpm(N, p, a)` (`analytical_solution/analytical_equations.py:4`) . . . 26
9.2 `shchukin_eq(n, p, a)` . . . 27
9.3 `bernardes_eq(n_segments, p)` . . . 27
9.4 `markov_approach(vertices, p, a)` and `including_fusion(vertices, p, a, f)` . . . 27
9.5 `n_segment_solution(n, pgen, pswap)` . . . 27
9.6 Monte-Carlo sampler (`analytical_solution/monte_carlo.py`) . . . 27
9.7 Utility helpers (`analytical_solution/utils.py`) . . . 27

10 Configuration and Canonical Baselines **28**
10.1 Phase-2 baseline ($n = 5$) . . . 28
10.2 Reported result and current Myriad archive state . . . 28
10.3 Myriad / qsub wrappers . . . 28

11 Output Layout **28**

12 Known Issues and Modeling Caveats **29**

13 Performance Levers **31**
13.1 In-loop hyperparameters worth sweeping . . . 31
13.2 Reward-shape candidates (require physics input) . . . 31
13.3 Action-space restructuring . . . 32
13.4 Curriculum and exploration . . . 32
13.5 Throughput . . . 32

14 File-by-File Reference **32**
14.1 `qamel/environment.py` . . . 32
14.2 `qamel/utils.py` . . . 32
14.3 `qamel/agent.py` . . . 33
14.4 `qamel/dqn.py` . . . 33
14.5 `scripts/train_qamel.py` . . . 33
14.6 `scripts/evaluate_qamel.py` . . . 33
14.7 `scripts/head_to_head.py` . . . 34
14.8 `scripts/run_stage2_study.sh` . . . 34
14.9 Myriad wrappers . . . 34
14.10 `scripts/local_pilot_stage2_lq_seed12345_progressive.sh` . . . 34
14.11 `analytical_solution/analytical_equations.py` . . . 34
14.12 `analytical_solution/monte_carlo.py` . . . 34
14.13 `analytical_solution/utils.py` . . . 34
14.14 Notebooks . . . 34
14.15 Top-level documents . . . 35

15 Glossary **35**

16 What to do first as the collaborating Claude **36**

1 Latest Implementation and Result Log (through June 19, 2026)

This log records the current state of the reinforcement-learning branch after the June 11, 2026 integration pass.

- Added an optional dueling DQN architecture, selected with `--dueling`. The original `DQNNet` remains the default. The dueling trunk now uses the same `net.1` and `net.3` state-dict keys as the original network trunk, so trunk weights can be copied from a plain DQN checkpoint into a dueling model by matching keys. A bug was found during this audit where the dueling trunk was named `trunk.*`; that made the checkpoint keys disjoint from `DQNNet`. This was fixed on June 11, 2026.
- Added optional Double DQN updates with `--double-dqn`. When enabled, the policy net selects the next valid action and the target net evaluates it.
- Added optional potential-based reward shaping with `--pbrs` and `--pbrs-scale`. The potential is $\Phi(s)$, the length of the longest contiguous run of elementary links starting at node 0. Training stores the shaped reward in replay but still logs the unshaped environment reward.
- Split replay-buffer terminal flags into `terminated` and `truncated`; truncated transitions now continue to bootstrap in the Bellman target. Legacy five-field replay entries are still loadable, but their old `done` flag cannot distinguish true termination from timeout.
- Added `scripts/head_to_head.py`, a seeded comparison harness for `dqn_greedy`, `dqn_swapprefer`, and a non-neural heuristic. It reads `config.json`, prefers `checkpoints/best_eval.pt`, writes JSON with per-seed summaries, and prints a Markdown table.
- Updated evaluation model construction to use the checkpoint/config architecture selector, so `evaluate_qamel.py` can load either plain or dueling DQN checkpoints.
- Current syntax/import checks pass for `qamel.dqn`, `qamel.utils`, `scripts.train_qamel`, `scripts.evaluate_qamel`, and `scripts.head_to_head`. No long training run was executed as part of this log.

1.1 Lab notebook entry: Myriad status on June 17, 2026

The cluster status check on UCL Myriad found no active queued/running jobs from `qstat -u "$USER"`. The submitted artifacts are therefore either completed, failed, or terminated by the scheduler. The relevant evidence came from `qamel/outputs/studies/*/cross_budget_summary.csv`, `qamel/outputs/cluster_logs/*.out`, and `qamel/outputs/stage0_escalation_logs/*.out`.

Run tag	Source	Budget	Success rate	Status
`lq_n5_seed12345_progressive`	Myriad archive	1,000	0.024	archived
`lq_n5_seed12345_progressive`	Myriad archive	5,000	0.506	archived
`lq_n5_seed12345_progressive`	Myriad archive	10,000	0.904	archived
`lq_n5_seed12345_progressive`	checkpoint dir	11,000	–	checkpoint exists, no eval archive
`lq_n6_seed12345_progressive`	Myriad archive	1,000	0.162	archived
`lq_n6_seed12345_progressive`	Myriad archive	2,000	0.012	archived
`lq_n6_seed12345_progressive`	Myriad archive	3,000	0.208	archived
`lq_n6_seed12345_progressive`	Myriad archive	4,000	0.000	archived
`swapaware_seed12345`	Myriad archive	200	0.204	archived, later resume failed

Interpretation:

- The strongest confirmed cluster result remains the single-seed $n = 5$, $p_{\text{gen}} = 0.4$, $p_{\text{swap}} = 0.7$ progressive run at 10,000 training episodes: success rate 0.904, mean return 58.618, mean steps 23.182, and truncated fraction 0.096 over a 500-episode greedy evaluation.
- The planned 20,000-episode continuation for that same $n = 5$ run did start and wrote `episode_011000.pt`, but no `budget_20000/eval_summary.json` was archived. The current lab-book state is therefore "10,000 complete; 20,000 incomplete", not "20,000 failed scientifically".
- The submitted Stage-0 escalation array `qamel/outputs/stage0_escalation_logs/qamel_s0_esc.612520.*.out` did not train. Every task exited at argument parsing with `unrecognized arguments: --max_actions 100 --double-dqn --dueling --pbrs --pbrs-scale 1.0 --lr 5e-4 --batch_size 256`. This is an operational failure: the Myriad checkout was stale and did not contain the newer training CLI flags.
- Later $n = 6$ progressive submissions attempted to resume an old checkpoint whose hidden width was 512, while the current $n = 6$ model width is 768 under the formula in §6. The resulting `size mismatch` is expected; $n = 6$ must be restarted under a fresh run tag or after moving the legacy checkpoint directory aside.
- The `swapaware_seed12345` run archived a 200-episode evaluation but then failed on resume because the stale cluster code tried to restore an RNG state that was not a `torch.ByteTensor`. The current local training code converts restored RNG tensors before calling `torch.random.set_rng_state`, but this must be confirmed on Myriad after a clean `git pull`.

Immediate operational conclusion: before any new Stage-0, dueling, Double-DQN, PBRS, or publication-suite job is submitted on Myriad, the cluster checkout must be made clean enough for `git pull --ff-only`. Otherwise the scheduler will keep running older scripts that reject the new flags.

1.2 Lab notebook entry: $n = 6$ no-go and $n = 7$ control status on June 19, 2026

The running Myriad arrays were inspected using both the rolling training proxy and every available greedy best-evaluation check. The greedy trajectories change the interpretation: the current $n = 6$ configuration is unstable rather than merely slow to learn. It is a **no-go for a full multi-seed claim**. The already-running tasks are pilots; held seeds should not be released under this configuration.

Seed	Greedy evaluation trajectory	Interpretation
202	0.03 → 0.62 (6k) → 0.51 (8k) → 0.25 (10k)	learn then collapse
303	stored peak 0.68 (4k); 0.025 → 0.020 → 0.005 → 0.015 (10k–16k)	early peak then near-dead
404	0.08 → 0.63 (4k)	healthy so far, still in danger window

For seed 202, the rolling training proxy had also fallen to 0.005 at episode 10,400, confirming that the live policy was nearly dead after its 0.62 greedy peak. The frozen `best_eval.pt` protects the peak artifact from subsequent degradation. Seed 303 likewise had an early stored 0.68 checkpoint, but its later greedy checks stayed around 1–2% with mean steps near 98, indicating near-constant truncation. This corrects the initial shorthand that seed 303 "never learned": the complete evidence

instead shows a still stronger learn-then-collapse failure. Seed 404 had reached 0.63 at episode 4,000, but seed 202 was similarly healthy at that age before failing. The current read is therefore two collapsed seeds and one not-yet-failed seed, versus the clean 8/8 $n = 5$ Stage-0 result.

Working diagnosis

The dominant configuration problem is the exploration schedule. In `scripts/train_qamel.py`, `eps_decay_steps` is measured in environment steps through `steps_done`, not in episodes. The $n = 6$ array sets it to only 30,000 even though the full run is of order 1.3 million or more environment steps; seed 202 had already accumulated roughly 133k steps by episode 4,000. Epsilon consequently reaches its 0.05 floor by roughly episode 900 and stays there for about 98% of training. A degraded policy therefore has almost no exploratory mechanism with which to recover.

The remaining stability settings plausibly amplify this failure at the larger $n = 6$ network/action scale: the array uses a 768-wide network, learning rate 5×10^{-4}, hard target synchronisation every 1,000 environment steps, and gradient clipping at 10.0. The observed early rise followed by a sharp, persistent drop is consistent with catastrophic forgetting and Q-value overestimation or moving-target oscillation. This is a working diagnosis, not a separately instrumented Q-value measurement.

There is also a measurement/configuration bug. The wrappers set the training and final gate horizon to 150 actions for $n = 6$ and 200 for $n = 7$, but do not pass `--best_eval_max_actions`; checkpoint selection therefore uses its default horizon of 100. This explains failing greedy checks with mean steps near 98 and means that "best" checkpoint selection is not aligned with the final head-to-head gate.

The training horizon itself is *not* the indicated lever. Seed 202's 0.62 peak had mean steps 49, showing that the chain is solvable well inside the existing limit. Keep `MAX_ACTIONS=150`, the curriculum, and the 40k episode budget; align best-eval with that horizon rather than increasing it.

Recommended fresh-tag retune

If the diagnosis is confirmed, terminate the current $n = 6$ pilots and relaunch fresh tags with `--force_train`; do not resume these unstable checkpoints. Apply changes in this priority order:

1. Increase `EPS_DECAY` from 30,000 to approximately 250,000 environment steps so exploration decays over roughly the first 20% of the run rather than its opening few percent.
2. Lower the learning rate from 5×10^{-4} to 2×10^{-4}.
3. Pass `--best_eval_max_actions 150` so checkpoint selection and the final $n = 6$ gate use the same horizon.
4. Optionally increase hard target synchronisation from 1,000 to 2,000 environment steps to damp oscillation; this is secondary to exploration and learning-rate changes.

One cheap confirmation remains available before terminating tasks: allow seed 404 to reach roughly episodes 8k–10k. A peak near 0.62 followed by the same drop would reproduce the failure naturally. This confirmation is useful but does not change the retune path.

The $n = 7$ flat-head run remains a deliberate weak control over roughly 233 unfactored actions. Its episode-12k candidate for seed 202 was 0.215, while the stored earlier best was 0.615 at episode 9k. To make the control scientifically clean, it should receive the analogous slower epsilon decay,

lower learning rate, and `--best_eval_max_actions 200`; otherwise weak performance can be attributed to under-tuning rather than to the flat action head. This was the state at diagnosis time; the follow-up implementation below records the applied wrapper changes and operational response.

Follow-up action and v2 implementation record

The operational response reported from Myriad was to preserve the $n = 6$ seed-404 task as a free confirmation through approximately episodes 8k–10k, delete the collapsed seed-202 and seed-303 tasks plus pending $n = 6$ v1 work, and delete the $n = 7$ v1 control array. These scheduler actions are user-reported in this local overview; they cannot be independently reconstructed from the repository.

The corresponding wrapper retune is verified in commit `a1cd14e` (present on `main` and `origin/main`). The implementation changes are:

Lever	$n = 6$ v2	$n = 7$ v2
`EPS_DECAY`	30k → 250k	45k → 450k
`LR`	$5 \times 10^{-4} \rightarrow 2 \times 10^{-4}$	$5 \times 10^{-4} \rightarrow 2 \times 10^{-4}$
Best-eval horizon	100 → 150	100 → 200
Run tag	`stage0_fullstack_n6_v2_s*`	`flathead_n7_v2_s*`

Both wrappers now expose `BEST_EVAL_MAX_ACTIONS` as an overridable tunable, pass it explicitly to `--best_eval_max_actions`, and print it with epsilon decay and learning rate in the initial canary diagnostics. The fresh v2 run tags, combined with `--force_train`, prevent collision with or accidental resume from v1 and preserve the frozen v1 `best_eval.pt` artifacts.

The retune intentionally leaves the training horizon, three-phase curriculum, episode budgets, architecture, PBRS, Double-DQN, dueling head, batch size, and hard target-update interval unchanged. The training-time prefer-swap option also remains off. This isolates the slower exploration decay, lower learning rate, and corrected checkpoint-selection horizon as the diagnosed changes. A 2,000 step target interval is not included because the interval is currently fixed at 1,000 in `scripts/train_qamel.py`, with no CLI override.

The relaunch protocol is deliberately sequential:

1. Pull commit `a1cd14e` on Myriad and submit only $n = 6$ array task 1 as the v2 canary.
2. Near episode 2k, verify that the logged epsilon remains materially above 0.05. Reaching the floor there means the intended schedule is not active.
3. Near episodes 10k–12k, require the greedy best-eval trajectory to remain high rather than repeating the v1 peak-then-collapse pattern.
4. Release $n = 6$ tasks 2–8 only after the canary survives that collapse window. Re-canary $n = 7$ afterward rather than launching its full array first.

If v2 still collapses despite slower exploration and the lower learning rate, the next controlled levers are training-time prefer-swap and a configurable softer target update. A repeated failure would promote the factored action head from an $n = 7$ scaling improvement to the primary path for both $n = 6$ and $n = 7$.

1.3 Lab notebook entry: Myriad workflow on June 17, 2026

This is the current working Myriad checklist. It duplicates the practical note in `MYRIAD_WORKFLOW.md` so that the CodeOverview PDF is self-contained. It is an operating procedure, not a scientific result.

Login and repository

```
ssh ucapgoz@myriad.rc.ucl.ac.uk
cd /home/ucapgoz/Timing-Multiuser-Protocols
```

All qsub wrappers assume:

```
/home/ucapgoz/Timing-Multiuser-Protocols
```

Module and virtual-environment setup

Two virtual environments were found:

```
/home/ucapgoz/venvs/timing-multiuser
/home/ucapgoz/venvs/dqn-env
```

Use `/home/ucapgoz/venvs/timing-multiuser`. Do not use `/home/ucapgoz/venvs/dqn-env` for current jobs; it failed with:

```
python: error while loading shared libraries: libpython3.11.so.1.0
```

Important ordering: `timing-multiuser` only sees PyTorch after the Myriad PyTorch module is loaded. Activating the venv first in a plain login shell led to `ModuleNotFoundError: No module named 'torch'`.

Known-good setup sequence:

```
deactivate 2>/dev/null || true

module unload compilers mpi gcc-libs 2>/dev/null || true
module load gcc-libs/10.2.0
module load python3/3.9-gnu-10.2.0
module load pytorch/2.1.0/gpu

source /home/ucapgoz/venvs/timing-multiuser/bin/activate
```

Validate the environment before submitting:

```
which python
python -V
python -c "import torch; print(torch.__version__); print(torch.cuda.is_available())"
python -c "import qamel.dqn; import qamel.utils; import scripts.head_to_head; print('imports ok
```

On the login node, `torch.cuda.is_available()` can print `False`; this is expected. In a GPU qsub job, the log should print `cuda_available True`. The verified login-node smoke check on June 17, 2026 was:

```
Python 3.9.6
torch 2.1.0+cu121
imports ok
```

Git hygiene before qsub

Before submitting cluster jobs, make sure Myriad is actually running the current code:

```
git status --short
git pull --ff-only
python -c "import qamel.dqn; import qamel.utils; import scripts.head_to_head; print('imports ol
```

If the checkout is dirty, the Git version on Myriad may reject `git stash push -u`. Use the older syntax:

```
git stash save -u "myriad pre-pull $(date +%Y%m%d-%H%M%S)"
git pull --ff-only
```

This matters: the first Stage-0 escalation array failed because Myriad still had an older `scripts/train_qamel.py`. The stale script rejected:

```
--max_actions 100 --double-dqn --dueling --pbrs --pbrs-scale 1.0 --lr 5e-4 --batch_size 256
```

Treat cluster results as untrusted unless the log proves the job ran after a clean pull/import check.

Submitting jobs

Stage-0 escalation array:

```
qsub scripts/myriad_stage0_escalation_array.sh
qstat -u "$USER"
```

This array uses task IDs 1–8 and writes logs to:

```
qamel/outputs/stage0_escalation_logs/
```

Expected early GPU-job log lines:

```
cuda_available True
device_count 1
device_name NVIDIA A100 80GB PCIe
```

Publication suite:

```
EXPERIMENT_SET=n5_canonical qsub scripts/qsub_publication_suite.sh
EXPERIMENT_SET=n6_prelim qsub scripts/qsub_publication_suite.sh
EXPERIMENT_SET=n5_dueling_double qsub scripts/qsub_publication_suite.sh
EXPERIMENT_SET=n5_dueling_double_pbrs qsub scripts/qsub_publication_suite.sh
```

The publication qsub wrapper delegates to `scripts/myriad_publication_suite.sh` and `scripts/run_stage2_study` and writes logs to:

```
qamel/outputs/myriad_logs/
```

Older Stage-2 wrappers still exist:

```
qsub scripts/qsub_myriad_stage2_lq_n5_seed12345_progressive.sh
qsub scripts/qsub_myriad_stage2_lq_n6_seed12345_progressive.sh
qsub scripts/qsub_myriad_stage2_swapaware_seed12345.sh
```

Use these only when intentionally reproducing the older progressive-study workflow. They write to `qamel/outputs/cluster_logs`.

Monitoring jobs and logs

```
qstat -u "$USER"

find qamel/outputs -maxdepth 3 -type d \( -name "*log*" -o -name "*logs*" \) -print

tail -n 120 qamel/outputs/stage0_escalation_logs/*.out 2>/dev/null
tail -n 120 qamel/outputs/myriad_logs/*.out 2>/dev/null
tail -n 120 qamel/outputs/cluster_logs/*.out 2>/dev/null
```

Search for high-signal lines:

```
grep -RInE "cuda_available|Success rate|Cross-budget|verdict|Traceback|error|size mismatch|unr
  qamel/outputs/stage0_escalation_logs qamel/outputs/myriad_logs qamel/outputs/cluster_logs 2>
```

Print archived Stage-2 budget summaries:

```
for f in qamel/outputs/studies/*/cross_budget_summary.csv; do
  echo
  echo "== $f =="
  cat "$f"
done
```

Known Myriad results and failures

- **Confirmed $n = 5$ progressive result.** `lq_n5_seed12345_progressive` reached success rate 0.904 at budget 10,000, with mean return 58.618, mean steps 23.182, truncated fraction 0.096, and 500 evaluation episodes. The same run has `episode_011000.pt`, but no archived `budget_20000` evaluation. Cite it as 10k complete, not 20k complete.
- **Stage-0 array failure.** The first Stage-0 escalation array did not test the science; it failed during argument parsing because the Myriad checkout was stale.
- **$n = 6$ checkpoint mismatch.** Some old $n = 6$ checkpoints use hidden width 512. The current $n = 6$ model with `counter_exposed_plus_ready` uses hidden width 768. Use a fresh run tag for new $n = 6$ jobs, or move the old run directory aside.
- **Swapaware resume failure.** The old `swapaware_seed12345` run archived a 200-episode eval but later failed restoring RNG state. Current local code converts restored RNG tensors before setting RNG state; confirm on Myriad after a clean pull before trusting resumed swapaware jobs.

Quick pre-submission checklist

```
cd /home/ucapgoz/Timing-Multiuser-Protocols

module unload compilers mpi gcc-libs 2>/dev/null || true
module load gcc-libs/10.2.0
module load python3/3.9-gnu-10.2.0
module load pytorch/2.1.0/gpu
source /home/ucapgoz/venvs/timing-multiuser/bin/activate
```

```
git status --short
git pull --ff-only

python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
python -c "import qamel.dqn; import qamel.utils; import scripts.head_to_head; print('imports ok
```

```
qstat -u "$USER"
```

2 Stage-0 Result Update (June 15, 2026)

The current Stage-0 result is a corrected *single-training-seed* result for the $n = 5$, $p_{\text{gen}} = 0.4$, $p_{\text{swap}} = 0.7$ repeater-chain task. It is strong enough to document the method and the physics interpretation, but it should not be presented as the final paper robustness claim until the 10-training-seed escalation finishes. The figure source is `notebooks/stage0_figures.ipynb`; the figures below are written under `figures/`.

Important June 17 lab-note caveat: the submitted Myriad Stage-0 escalation array did *not* test this scientific claim. It failed during CLI parsing because the cluster checkout was stale and did not yet contain the newer `--dueling`, `--double-dqn`, `--pbrs`, learning-rate, batch-size, and max-action flags. The failed array is therefore an operations/checkouts failure, not evidence that the Stage-0 method failed.

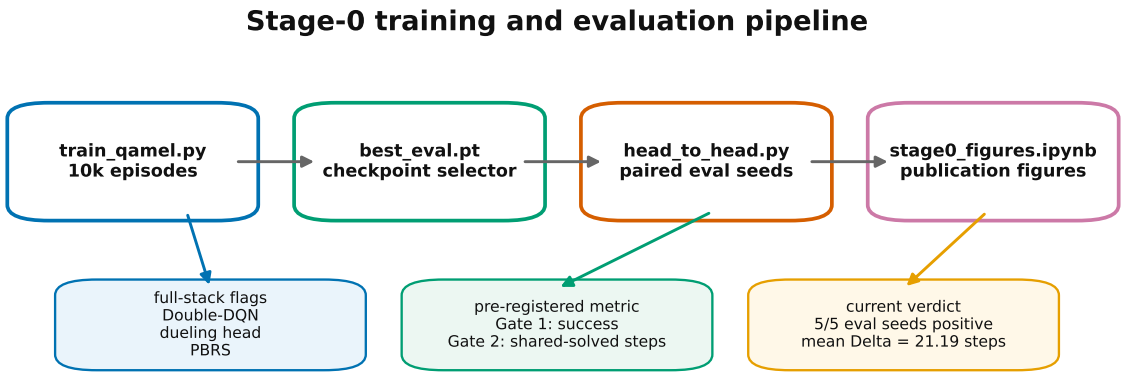


Figure 1: Stage-0 pipeline. Training produces a `best_eval.pt` checkpoint, the paired head-to-head harness evaluates the learned policy against the heuristic on identical stochastic instances, and `stage0_figures.ipynb` turns the locked artifacts into publication figures.

2.1 Stage-0 method in one pass

The full-stack Stage-0 configuration uses:

- `counter_exposed_plus_ready` observations: adjacency, generation counters, swap counters, and a derived swap-readiness channel.

- Dueling DQN: a shared trunk with separate value and advantage heads,

$$Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a').$$

- Double-DQN target updates: the policy network chooses the best valid next action while the target network evaluates that action.
- State-valid action masking both at acting time and inside the Bellman target.
- Potential-based reward shaping (PBRS) with

$$F(s,s') = \gamma\Phi(s') - \Phi(s),$$

  where $\Phi(s)$ is the longest contiguous run of elementary links starting at node 0. This is a learning aid, not a new physical evaluation metric.
- Correct separation of `terminated` and `truncated`: time-limit truncations continue to bootstrap, while true terminal states mask the target.

2.2 Stage-0 physics in one pass

The experiment models discrete control of a binary Bell-pair repeater chain. A generation request samples a nearest-neighbour Bell pair with probability $p_{\text{gen}} = 0.4$; a swap request at an interior node samples entanglement swapping with probability $p_{\text{swap}} = 0.7$. A successful swap consumes two shorter links and creates a longer link between their outer endpoints. The target event is a direct end-to-end link between the two boundary nodes. The model does not simulate fidelity, decoherence, finite memory lifetime, physical clock time, purification, classical signaling, or grid/multipartite fusion. Therefore the Stage-0 claim is about *control efficiency in discrete repeater-chain scheduling*, not about full physical latency.

2.3 Stage-0 numerical result

The head-to-head comparison follows the pre-registered metric in `METRIC.md`. It evaluates `dqn_greedy` and the non-neural prefer-swap-when-ready heuristic on the same 1000 stochastic episodes for each of five evaluation seeds: $12345, 23456, 34567, 45678, 56789$.

The decision rule has two ordered gates. Gate 1 requires non-inferior success relative to the heuristic within $\delta = 0.02$. Gate 2 restricts to the shared-solved episode set and computes

$$\Delta = \text{steps}_{\text{heuristic}} - \text{steps}_{\text{DQN}}.$$

A positive Δ means the DQN spans the same problem instance faster.

Metric	Stage-0 value
DQN greedy success rate	0.9874
Heuristic success rate	0.8550
DQN greedy mean steps	17.2091
Heuristic mean steps	38.1017
Shared-solved paired mean Δ	21.1889 steps
95% paired-t CI for mean Δ	[20.2454, 22.1325]
Positive evaluation seeds	5/5
Gate verdict	WIN

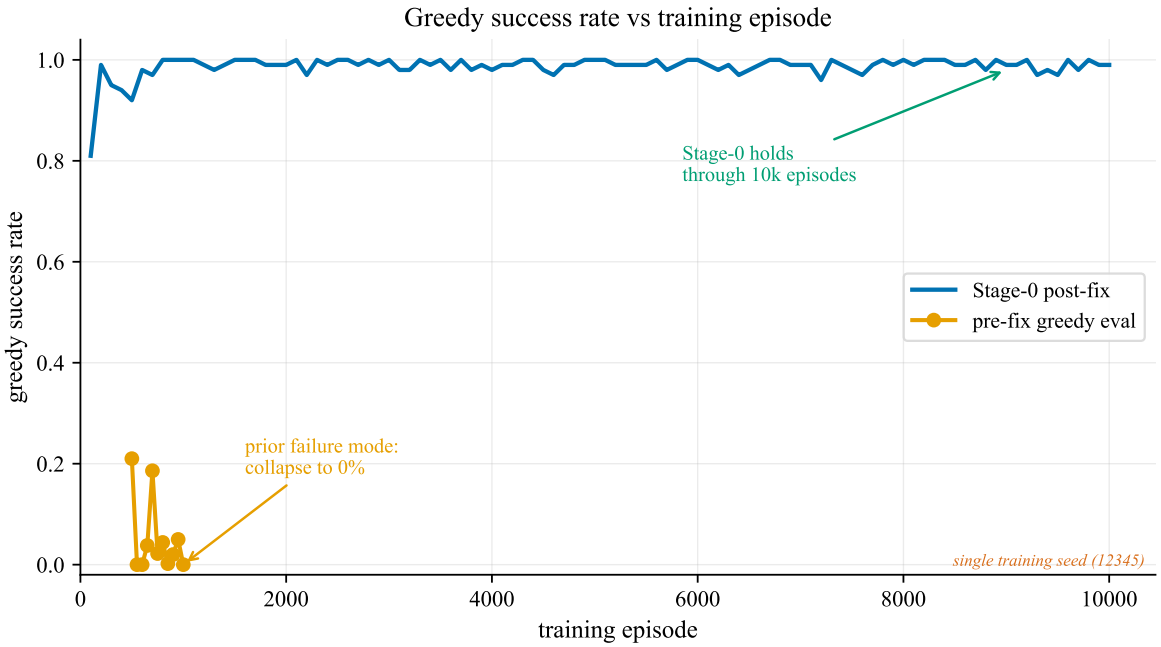


Figure 2: The corrected Stage-0 training path removes the late greedy-evaluation collapse seen in the pre-fix checkpoint sweep. Both traces are plotted on the same greedy-success-rate axis.

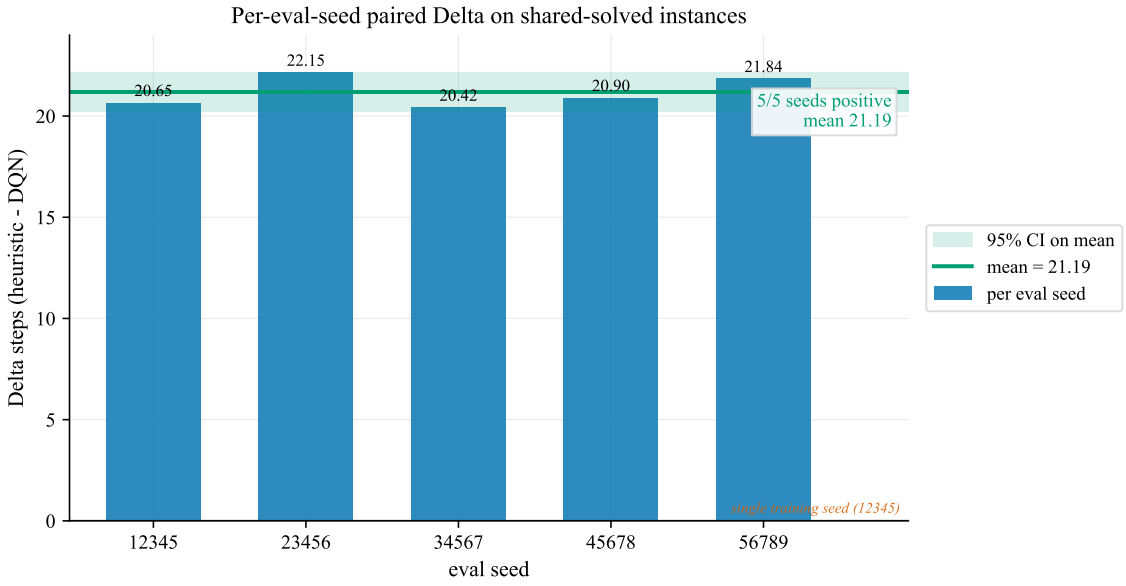


Figure 3: Per-evaluation-seed paired step advantage. On the shared-solved episode intersections, all five evaluation seeds have positive Δ = heuristic steps − DQN steps, with mean 21.19 steps.

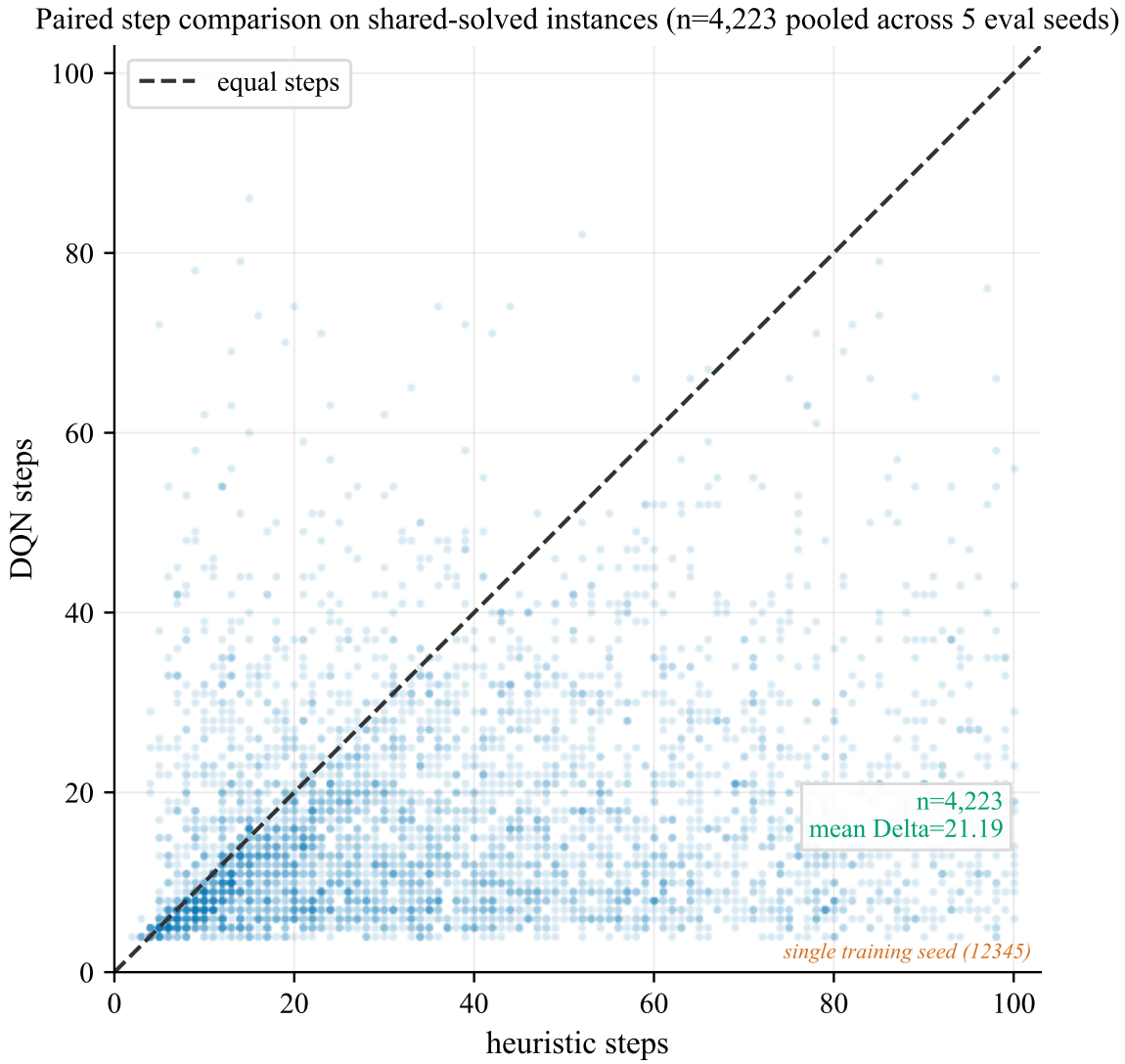
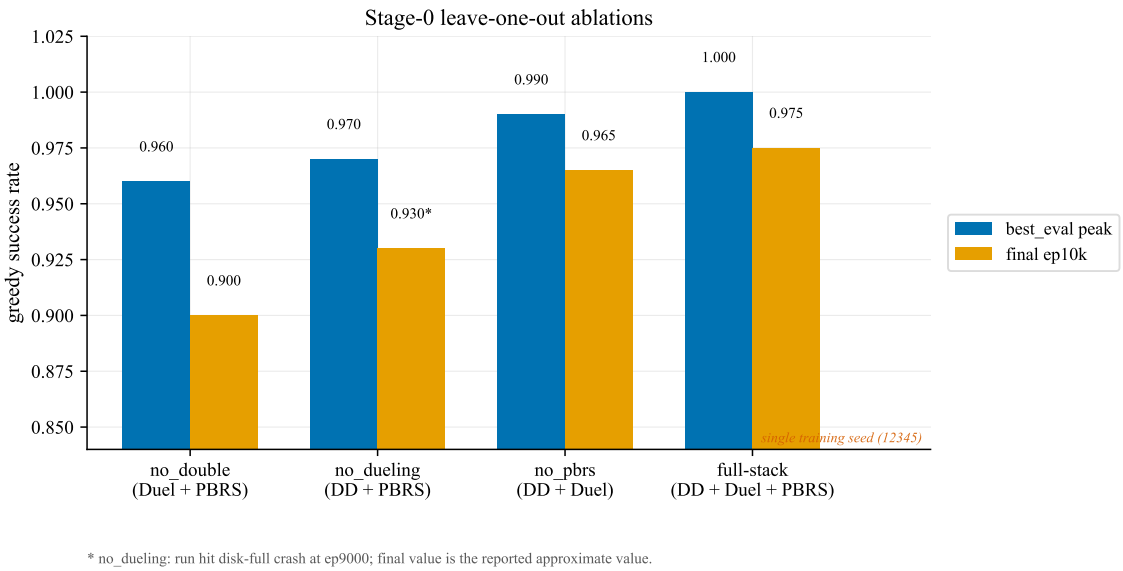


Figure 4: Episode-level paired scatter over the 4,223 pooled shared-solved instances. Points below the dashed $y = x$ diagonal are direct DQN step wins on identical stochastic problem instances.



* no_dueling: run hit disk-full crash at ep9000; final value is the reported approximate value.

Figure 5: Stage-0 leave-one-out ablations. Double-DQN is the largest stabilizer in this stack; dueling and PBRS add incremental value once the termination/truncation and target-masking fixes are in place.

3 Project Overview and Scope

3.1 What the repository *actually implements*

The executable RL code learns a control policy for a *one-dimensional quantum repeater chain*. It does not, at present, implement a 2D grid with multipartite (GHZ-class) distribution; that broader setting only appears in the analytical baseline files and in the OFC 2025 paper PDF (`OFC_paper_2025.pdf`).

Concretely:

- A chain of n nodes, indexed $0, 1, \ldots, n-1$. Nodes 0 and $n-1$ are the endpoints. Interior nodes are repeaters.
- Nearest-neighbour elementary links $(i, i+1)$ can be probabilistically generated with success probability p_{gen}.
- Interior repeater nodes can attempt entanglement swapping, succeeding with probability p_{swap}.
- Goal: produce a direct entangled link between node 0 and node $n-1$ using as few elementary operations / steps as possible.

The repository ships two RL solvers for this problem:

1. A **tabular Q-learning** baseline (`qamel/agent.py`) over enumerated adjacency states. This scales to roughly $n \leq 6$.
2. A **Deep Q-Network** (`qamel/dqn.py`, `scripts/train_qamel.py`). This is the workhorse for $n \in \{5, 6\}$ with curriculum learning.

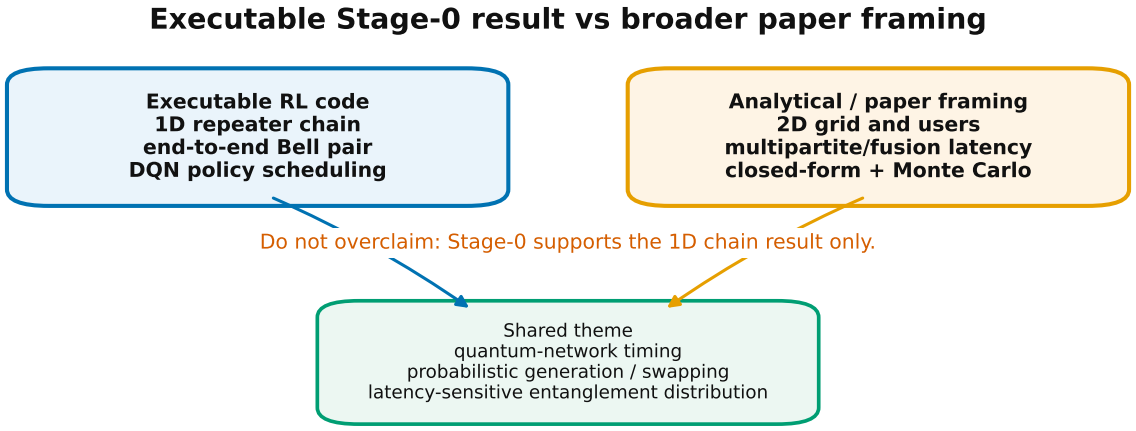


Figure 6: Scope map for collaborators. The executable Stage-0 result supports the 1D repeater-chain Bell-pair scheduler. The grid/multipartite/fusion story is present in the analytical and paper-framing layer but is not yet an RL environment.

3.2 What the analytical side implements (different problem!)

The files in `analytical_solution/` and the verification notebooks (`notebooks/verify_simulations.ipynb`, `notebooks/calculate_ratio_and_latencies.ipynb`) compute closed-form and Monte-Carlo waiting-time statistics for a *grid topology* with four users at the corners and a central node. They include fusion probability p_{fusion}. These functions are not currently wired into the RL environment; they sit alongside it as a reference benchmark.

3.3 Why the two scopes diverge

The OFC 2025 paper frames the work as multipartite entanglement distribution in a quantum software-defined networking (SDN) architecture, including fusion and latency budgeting. The RL code is a strict subproblem: bipartite end-to-end Bell-pair scheduling on a chain. Any physics-aware reviewer must keep this divergence in mind. See §12 for the canonical list of known mismatches.

4 Physical Model

4.1 Chain dynamics implemented in `RepeaterChain`

The class `RepeaterChain` (`qamel/environment.py:3`) is the entire executable physics model. It assumes:

Topology
: Linear chain of n nodes; only nearest-neighbour elementary links are allowed.

Link representation
: Binary; either an entangled link is present between two nodes or it is not. No fidelity, no Schmidt rank, no Werner parameter.

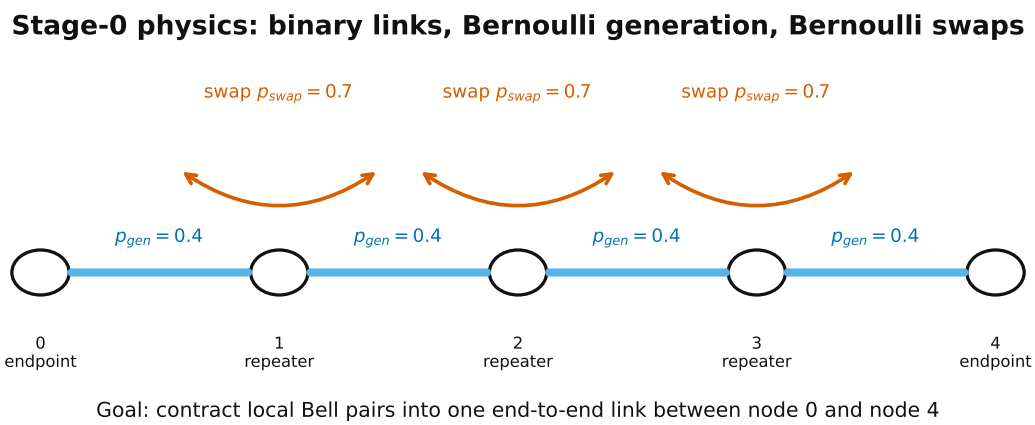


Figure 7: Physics abstraction used by Stage-0. The agent schedules elementary generation on nearest-neighbour segments and swaps at interior repeaters until the chain contracts into one end-to-end Bell pair.

Generation

On an action requesting an elementary edge $(i, i+1)$, the existing link is cleared, both endpoint generation counters are incremented, and the link is re-sampled with $\Pr = p_{\text{gen}}$.

Swapping

On an action requesting a swap at interior node k with exactly two currently-incident links (to neighbours j and ℓ), the swap counter at k is incremented and with $\Pr = p_{\text{swap}}$ a direct edge (j, ℓ) is created. The two links incident to k are then removed unconditionally.

Persistence

Links remain present until consumed by a swap or until the agent re-requests generation on that edge.

4.2 What is intentionally *not* modelled

For a physics collaborator, the most important omissions are:

- No memory lifetime / decoherence.
- No fidelity tracking. Entanglement is treated as ideal Bell pairs.
- No classical signaling latency. Each environment "step" is unitless.
- No teleportation latency or BSM duration.
- No fusion operations. (Fusion appears only in the analytical equations.)
- No noise model on swaps beyond the Bernoulli success/fail.

The exported metrics `ent_attempt_max` and `swap_attempt_max` are then post-processed by `notebooks/calculate_ra` into physical latencies under fixed operation-time and fibre-propagation assumptions. That post-processing is external to the RL environment.

5 MDP Formulation

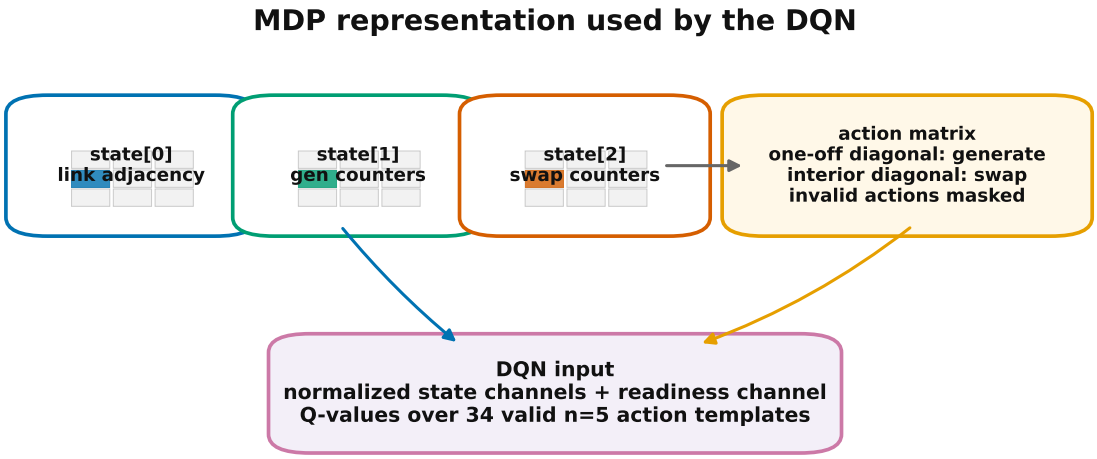


Figure 8: MDP representation. The physical state is stored as three $n \times n$ channels; Stage-0 adds a derived readiness channel before feeding the observation to the DQN. The action is also an $n \times n$ matrix, with generation encoded on one-off diagonals and swaps encoded on interior diagonal entries.

5.1 State $s_t \in \mathbb{R}^{3\times n\times n}$

`RepeaterChain.reset()` returns a $3 \times n \times n$ zero tensor (`qamel/environment.py:10`). The three channels are:

Channel	Meaning
$s[0]$	Symmetric binary adjacency matrix of current entangled links.
$s[1]$	Diagonal-only: per-node *generation attempt* counter.
$s[2]$	Diagonal-only: per-node *swap attempt* counter.

Counter channels are populated only on the diagonal. The off-diagonals of $s[1]$ and $s[2]$ remain zero throughout an episode.

5.2 Action $a_t \in \{0,1\}^{n\times n}$

An action is itself an $n \times n$ binary matrix encoding two simultaneous decisions:

- **Generation requests**: the one-offset diagonals $a[i, i+1]$ (and symmetrically $a[i+1, i]$) mark which elementary edges should be attempted this step. A generation request can "refresh" an existing link (clear it and re-sample).
- **Swap requests**: the diagonal entries $a[k, k]$ for interior nodes $k \in \{1, \ldots, n-2\}$ mark which repeater nodes should attempt a swap.

A *no-op* action is the zero matrix; it advances the step counter but does not touch the chain.

The complete set of admissible actions is enumerated once per n by `generate_all_valid_actions` (`qamel/utils.py`):

1. Form all matrices with arbitrary interior-diagonal and one-offset-diagonal bits, forcing endpoint diagonals to zero. There are $2^{(n-2)+(n-1)}$ such candidates (`qamel/utils.py`).
2. Reject any matrix where the row/column of a swap-marked node is nonzero off the diagonal. This enforces "do not generate a new edge on a node that you are also swapping this step."

Measured action-set cardinalities:

n	$\lvert\mathcal{A}_{\text{valid}}\rvert$
3	5
4	13
5	34
6	89
7	233

5.3 Transition $P(s_{t+1} \mid s_t, a_t)$

The transition is implemented as a single Python method, `RepeaterChain.step` (`qamel/environment.py:14`). In pseudocode:

```
clone state s -> s'

for each upper-triangular edge (i,j) with a[i,j]=1:
    s'[1][i,i] += 1                    # generation counter at i
    s'[1][j,j] += 1                    # generation counter at j
    s'[0][i,j] = s'[0][j,i] = 0        # clear edge
    if Bernoulli(pgen) succeeds:
        s'[0][i,j] = s'[0][j,i] = 1    # success creates the link

for each interior k with a[k,k]=1:
    neighbours = nonzero indices of s'[0][k]
    if len(neighbours) > 2:
        return (-100, s')              # ill-typed early exit (see issues)
    if len(neighbours) == 2:
        s'[2][k,k] += 1
        let (j, l) = neighbours
        carry = max(s'[0][j,k], s'[0][l,k])  # always 1 here
        if Bernoulli(pswap) succeeds:
            s'[0][j,l] = s'[0][l,j] = carry
        s'[0][j,k] = s'[0][k,j] = 0
        s'[0][l,k] = s'[0][k,l] = 0

return s'
```

Two subtle points:

- Generation requests are processed *before* swap requests, so a freshly generated link could in principle be swapped within the same step. However, `generate_all_valid_actions` forbids actions that simultaneously generate and swap at the same node, so the chain semantics are well defined.
- The branch on `len(neighbours) > 2` returns a tuple instead of a tensor. This is a latent bug (see §12, item 13).

5.4 Terminal conditions

Defined in `qamel/utils.py`:

- **Final / successful**: `check_if_final_state` (`qamel/utils.py`) returns true when $s[0, n-1] \neq 0$.
- **Bad**: `check_if_bad_state` (`qamel/utils.py`) returns true when any endpoint has degree > 1 or any interior node has degree > 2.
- **Truncated**: episode length reached `max_actions` (default 100).

`get_episode_status` (`qamel/utils.py`) is the canonical reducer: it considers a state "final" only if it is final *and not* simultaneously bad.

5.5 Reward

The reward is implemented in `reward_shape` and wrapped by `compute_reward` (`qamel/utils.py`):

$$r_t = \begin{cases} +100 & \text{if terminated (final and not bad)} \\ -100 & \text{if bad or truncated} \\ -1 & \text{otherwise} \end{cases}$$

Currently `reward_mode="base"` is the only supported mode; the function signature contains `swap_ready_bonus` for backward compatibility, but the bonus is not applied in the current canonical reward path.

Historical note. The original `REPORT.md` describes a terminal reward of $1000/\max(s)$ and a separate training-time `swap_ready_bonus`. Both have since been removed in favour of the simpler $\pm 100/-1$ scheme shown above. The codebase still exposes the `--swap_ready_bonus` flag for command-line compatibility, but `compute_reward` now ignores it.

5.6 Valid-action mask

`is_action_valid_given_state` (`qamel/utils.py`) computes a state-conditional validity check used both during action selection and during DQN target backups:

1. For each requested new edge in the upper triangle, increment a hypothetical post-action degree at each endpoint.
2. Endpoints have a hard cap of 1; interior nodes have a cap of 2.
3. Additionally, any node marked for swap must end up with post-action degree exactly 2 (otherwise the swap is meaningless or invalid).

This is a more refined check than the structural `generate_all_valid_actions` step: it depends on the current state, not just on the action layout.

6 The Reinforcement Learning Code

6.1 Tabular Q-learning (baseline)

File: `qamel/agent.py`. The `Agent` class:

- Enumerates all valid adjacency states via `generate_all_valid_states` (`qamel/utils.py`). Caches them under `qamel/outputs/logs/states/{n}_nodes.npy`.
- Enumerates all valid actions and caches them under `qamel/outputs/logs/actions/{n}_nodes.npy`. These are also reused by the DQN code.
- Holds a $|\mathcal{S}_{\text{valid}}| \times |\mathcal{A}_{\text{valid}}|$ Q-table.
- Exploration in `predict_action` (`qamel/agent.py:50`) is intended to be ε-greedy but currently uses `torch.randn(1) < epsilon` — a standard-normal comparison rather than a uniform one. This is a known bug.
- Terminal update in `update_q_table` (`qamel/agent.py:60`) adds Q instead of subtracting it, which is also incorrect.

For physics-side collaboration this learner can be treated as a smoke-test only; the real benchmark is the DQN.

6.2 DQN: observation modes

File: `qamel/dqn.py`. `preprocess_obs(state, obs_mode, counter_norm)` produces the network input.

`obs_mode`	Tensor shape	Contents
`baseline`	(n, n)	Adjacency only (used by tabular learner).
`counter_exposed`	$(3, n, n)$	Adjacency + normalised gen/swap counters.
`counter_exposed_plus_ready`	$(4, n, n)$	The above + global swap-readiness channel.

The counter normalisation clamps $s[1], s[2]$ to $[0, 1]$ after dividing by `counter_norm` (default 20). The swap-readiness channel is a constant plane equal to the count of interior nodes with current degree 2, providing a coarse "ready to swap somewhere" signal that survives convolution-free preprocessing.

6.3 DQN: network

`DQNNet` (`qamel/dqn.py`) is the default flat MLP:

```
Flatten -> Linear(input, H) -> ReLU
       -> Linear(H, H)       -> ReLU
       -> Linear(H, |A|)
where H = max(512, ceil(4 * input / 256) * 256)
```

The hidden width formula scales with the flattened observation dimension. For $n = 5$, `counter_exposed_plus_read` has input dim 100, so $H = 512$. For $n = 6$ the input dim is 144 and the formula yields $H = 768$ (since $4 \times 144 = 576$ rounds up to the next 256-multiple, 768).

`DuelingDQNNet` is also available through `build_dqn_net(..., net_arch="dueling")` and through the training flag `--dueling`. It uses the same flatten/two-layer trunk and then separate value and advantage heads:

$$Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a').$$

The trunk is intentionally named `net.1` and `net.3`, matching `DQNNet`'s first two linear layers. This allows partial trunk-weight transfer from a plain DQN checkpoint into a dueling checkpoint by matching state-dict keys; only the final plain-DQN output layer is architecture-specific.

6.4 DQN: training loop

The full DQN training loop is in `train_dqn_agent` (`scripts/train_qamel.py:558`). The structure, in order:

1. Build `RepeaterChain` environment.
2. Load (or generate and cache) the enumerated valid-action tensor.
3. Identify the no-op action index for fallback.
4. Build policy and target networks (`DQNNet` or `DuelingDQNNet`), Adam optimiser, replay buffer.
5. Optionally restore from a checkpoint (`resume_checkpoint`) including replay buffer, RNG state, optimiser state, and episode-level metric arrays.
6. Episode loop:
   (a) Optionally adjust the per-episode action horizon via the curriculum (see §6.6).
   (b) At each step:
   - Compute the linear ε from `linear_schedule` (`qamel/utils.py`).
   - Compute valid action indices for the current state.
   - Optionally restrict to swap-only actions when at least one interior node is swap-ready and `--prefer_swap_when_ready_train` is set.
   - ε-greedy choice over the restricted index set, with invalid Q-values masked to -10^9.
   - Step the environment, compute unshaped environment reward via `compute_reward`.
   - If `--pbrs` is enabled, compute $F = \gamma\Phi(s') - \Phi(s)$ and push $r_{\text{train}} = r_{\text{env}} + \lambda F$ into replay, where λ is `pbrs_scale`. Episode metrics still use r_{env}.
   - Push transition into the replay buffer (CPU tensors), storing `terminated` and `truncated` separately.
   - If buffer is large enough, sample a batch, build the Bellman target, mask next-state invalid actions *too* (`_build_valid_action_mask`, `scripts/train_qamel.py`), compute Smooth-L1 (Huber) loss, optimise with gradient clipping at $\|g\| \leq 10$. If `--double-dqn` is enabled, the policy net selects the next action and the target net evaluates it.
   - Periodically refresh the target network from the policy network.
   (c) At end of episode: record return, steps, success flag, mean per-step loss, mean swap-readiness.
   (d) Optionally save a checkpoint at `--checkpoint_every`, emit a metrics row at `--log_every`.

(e) Optionally run an offline "best-eval" rollout (§6.7) and update the best-eval checkpoint if the snapshot beats the running best on (success rate, mean return, lower mean steps) lex order.

7. Return the final tuple of (policy net, target net, optimiser, replay buffer, episode arrays, global step count, completed episodes, best-eval metrics).

6.5 DQN hyperparameters (canonical)

From `dqn_hyperparameters` (`scripts/train_qamel.py:79`):

Parameter	Value
γ (discount)	0.99
Learning rate	10^{-3}
Batch size	64
Replay buffer capacity	50,000
Target network update period (env steps)	1,000
$\varepsilon_{\text{start}}$	1.0
ε_{end}	0.05
ε-decay steps	10,000
Counter normalisation c_{norm}	20.0
Loss	Smooth-L1 (Huber)
Optimiser	Adam
Grad clip	$\|g\|_2 \leq 10$

The CLI can override the learning rate and batch size with `--lr` and `--batch_size`. Architecture and target-update variants are controlled by `--dueling` and `--double-dqn`; reward shaping is controlled by `--pbrs` and `--pbrs-scale`.

6.6 Curriculum learning

With `--use_curriculum`, the training loop divides the episode budget into three phases by `--curriculum_boundari` (default 5,000; 10,000) and uses progressively larger per-episode action horizons from `--curriculum_steps` (default $20, 40, 100$). Each time the phase rolls over, a one-line status print announces the new horizon.

6.7 Best-eval checkpointing

When `--best_eval_every` > 0, every k episodes the training loop:

1. Snapshots the RNG state.
2. Reseeds with `--best_eval_seed`.
3. Switches the policy net to eval mode and runs `--best_eval_episodes` rollouts at $\varepsilon = 0$, masking invalid actions but *not* applying the prefer-swap-when-ready filter.
4. Computes (success_rate, timeout_rate, mean_return, mean_steps, mean_ent_attempt_max, mean_swap_attemp
5. If the lexicographic comparator `_is_better_eval_metrics` prefers the candidate, the best-eval checkpoint and JSON are updated.

6. Restores the prior RNG state to avoid biasing the training stochastics.

This is the primary path for "frozen, reproducible" policy selection during long runs.

6.8 Resume semantics

On startup the training script searches for, in this order, the latest checkpoint at `qamel/outputs/runs/<run_name>/checkpoints/latest.pt`, then a legacy checkpoint location, then the final model file. If a checkpoint is found and `--force_train` is *not* set, training resumes from it: model weights, optimiser state, replay buffer contents, RNG state, episode arrays, and the global step count are all restored. The validator `_validate_resume_checkpoint` (`scripts/train_qamel.py`) refuses to resume if n, p_{gen}, p_{swap}, `obs_mode`, `reward_mode`, `swap_ready_bonus`, `prefer_swap_when_ready_train`, `seed`, or `net_arch` differ from those baked into the checkpoint.

7 Evaluation Pipeline

The script `scripts/evaluate_qamel.py` is the canonical offline checkpoint benchmark. It builds the network through `build_dqn_net`, so checkpoints with `net_arch="dqn"` and `net_arch="dueling"` are both loadable.

7.1 Policy execution

For each evaluation episode:

- Reset the environment.
- Loop steps until terminal/truncated.
- At each step, compute the validity mask, optionally apply ablation filters (§7.2), choose an action greedily (or ε-soft if `--eval_epsilon` > 0), step the environment, accumulate reward.
- Record episode-level statistics.

7.2 Ablation filters

Three filters can be combined on the valid-action set:

`--mask_null_action` Forbids the no-op when other actions are valid.

`--block_refresh_actions` Disallows actions whose generation requests overlap a currently-active edge. This prevents the policy from "wasting" a generation attempt to refresh a link that is already up.

`--prefer_swap_when_ready` If any interior node has post-action degree 2 (i.e. is ready to swap), restrict the candidate set to actions that include at least one swap bit.

If filtering reduces the set to empty, the loop transparently falls back to the unfiltered valid-action set and increments `ablation_empty_fallbacks` in the summary JSON, so the experimenter can detect when ablations are being silently ignored.

7.3 Tracing

With `--trace_max_episodes` > 0, a structured per-step JSON trace is emitted. Each step records: pre-state summary (active links, swap-ready node count, ent/swap attempt maxima), action summary (generation edges, swap nodes, refresh edges, null-action flag), reward, post-state summary, and status flags. With `--trace_failed_only` the trace is filtered to episodes that did not end in a successful final state. The trace lives at `--trace_output_path`.

7.4 Artifacts

For each evaluation invocation the script writes:

- `qamel/outputs/runs/<run_name>/evaluations/<eval_name>.csv` (per-episode rows).
- `..._summary.json` with the aggregate metrics.
- `..._ent_counts.txt`, `..._swap_counts.txt` for the count distributions.
- A legacy global CSV `qamel/outputs/results/eval_n<n>_pgen<p>_pswap<q>_<tag>.csv` for backwards compatibility with the notebooks.

The `run_name` is built deterministically as `dqn_n<n>_pgen<p>_pswap<q>_<tag>`; `eval_name` is `eval_episodes<E>_max<M>_eps<eps>_seed<s>` when a seed is supplied.

7.5 Head-to-head policy comparison

`scripts/head_to_head.py` compares three policies on the same environment configuration read from a run directory's `config.json`:

`dqn_greedy` DQN argmax over the state-conditional valid-action mask, with no prefer-swap filter.

`dqn_swapprefer` The same network, but if an interior node is currently degree 2, it restricts valid actions to those swapping at least one ready node; if the restriction is empty it falls back to the full valid set.

`heuristic` No network. If a ready interior node exists, choose uniformly among valid actions that swap a ready node. Otherwise choose uniformly among valid generation-only actions, avoiding the null action unless forced.

For each policy and seed it records success rate, mean steps on successful episodes, mean return, truncated fraction, and success count. The JSON output contains `config`, `per_run`, `summary`, and `verdict`. The `BEAT` verdict requires the mean greedy-DQN success rate to exceed the heuristic mean and at least four positive paired seed deltas under the default five-seed protocol.

8 Multi-Budget Study Pipeline

The script `scripts/run_stage2_study.sh` is the canonical sweep driver for cluster (UCL Myriad) runs.

8.1 Inputs (environment variables)

`N`, `PGEN`, `PSWAP`, `OBS_MODE`, `REWARD_MODE`, `EVAL_EPISODES`, `MAX_ACTIONS`, `EVAL_EPSILON`, `SEED`, `RUN_TAG`, `BUDGETS`, `CHECKPOINT_EVERY`, `LOG_EVERY`, `BEST_EVAL_EVERY`, `BEST_EVAL_EPISODES`, `BEST_EVAL_MAX` `BEST_EVAL_SEED`, `COUNTER_NORM`, `SWAP_READY_BONUS`, `PREFER_SWAP_WHEN_READY_TRAIN`, `USE_CURRICULUM`, `STUDY_ROOT`, `ALLOW_ARCHIVE_OVERWRITE`, `STUDY_FORCE_TRAIN`. Defaults match the canonical phase-2 preset.

8.2 Loop structure

For each budget B in `BUDGETS` (default $10000, 50000, 100000, 200000$):

1. Refuse to overwrite a non-empty archive directory unless `ALLOW_ARCHIVE_OVERWRITE=1`.
2. Call `train_qamel.py` with `--train_episodes=B`. Because of the resume logic, this picks up where the previous budget left off (so 50000 is an additional 40000 episodes after 10000, not from scratch). `STUDY_FORCE_TRAIN=1` only injects `--force_train` when the run directory does not yet exist.
3. Call `evaluate_qamel.py` at that budget.
4. Assert that all the expected artifacts exist.
5. Copy them under `qamel/outputs/studies/<RUN_TAG>/budget_<B>/`.
6. Rebuild `cross_budget_summary.csv` from the archived JSON summaries.

8.3 Cross-budget summary

The CSV columns are: `budget, success_rate, mean_return, mean_steps, mean_ent_attempt_max, mean_swap_attempt_max, truncated_fraction`. This is the main quantitative deliverable of a stage-2 study.

9 Analytical and Monte-Carlo Baseline

The directory `analytical_solution/` contains a self-contained *grid-topology* model used for paper figures and notebook verification. It is not currently coupled to the RL environment.

9.1 `generate_tpm(N, p, a)` (`analytical_solution/analytical_equations.py:4`)

Builds a transition probability matrix for an N-segment chain by recursive Kronecker doubling, following Shchukin et al. For each segment, the elementary TPM is

$$T_1 = \begin{pmatrix} 1-p & p \\ 0 & 1 \end{pmatrix},$$

i.e. a Bernoulli generation per time step with absorbing "link present" state. The recursion repeatedly takes Kronecker products of pairs and applies a swap-failure redistribution:

$$T_{2k}[:-1, 0] \mathrel{+}= (1-a)\, T_{2k}[:-1, -1], \qquad T_{2k}[:-1, -1] \mathrel{*}= a,$$

where $a \in \{p_{\text{swap}}, p_{\text{fusion}}\}$ depending on the caller. This implements "swap fails $\Rightarrow$ restart both halves" as a single absorbing step.

9.2 `shchukin_eq(n, p, a)`

Returns the expected number of generation attempts to produce an entangled link spanning n segments using the standard fundamental-matrix identity

$$\bar{K} = (I - Q)^{-1}\mathbf{1},$$

where Q is the substochastic block of T.

9.3 `bernardes_eq(n_segments, p)`

Implements the Bernardes closed form for non-deterministic generation:

$$E[T] = \sum_{j=1}^{n} (-1)^{j+1} \binom{n}{j} \frac{1}{1 - (1-p)^j}.$$

Valid only when generation is the rate-limiting step (no swap retries).

9.4 `markov_approach(vertices, p, a)` and `including_fusion(vertices, p, a, f)`

These specialise to a $V \times V$ grid with users at the four corners and a central node. The maximum Manhattan distance to the central node is computed, and `shchukin_eq` (or its fusion-corrected variant) is evaluated at that distance. This is the analytical waiting-time model used in the paper figures.

9.5 `n_segment_solution(n, pgen, pswap)`

Returns the full vector $\bar{K}$ over all initial sub-states of an n-segment chain TPM. Useful when one wants the conditional expectations starting from a partially-built chain.

9.6 Monte-Carlo sampler (`analytical_solution/monte_carlo.py`)

`MonteCarloFusion` samples waiting times by recursively realising the nested-swap tree:

- Level 0: `np.random.geometric(pgen)` for an elementary link.
- Level $n > 0$: sample two level-$(n-1)$ subtrees, take the max (both halves must be ready), increment the swap counter `tq`, and either accept with $\Pr = p_{\text{swap}}$ or recursively retry both halves.
- Fusion is applied multiplicatively: a separate geometric-distributed fusion attempt count is averaged in via `sample_fusion`.

The top-level helper `monte_carlo` returns the four-user joint waiting-time samples using the grid distances computed by `decompose_into_powers` (`analytical_solution/utils.py:28`).

9.7 Utility helpers (`analytical_solution/utils.py`)

`manhattan_distance`, `largest_power_of_2`, `decompose_into_powers`, `normalize_array`, plus `monte_carlo_sim` which is a thin wrapper for batched waiting-time sampling with a rich progress bar.

10 Configuration and Canonical Baselines

10.1 Phase-2 baseline ($n = 5$)

The canonical preset is `configs/phase2_baseline_n5_plus_ready.json`:

```
{
  "name": "phase2_baseline_n5_plus_ready",
  "train": {
    "n": 5, "pgen": 0.4, "pswap": 0.7,
    "obs_mode": "counter_exposed_plus_ready",
    "seed": 12345, "reward_mode": "base",
    "swap_ready_bonus": 0.5,
    "train_episodes": 10000, "max_actions": 100,
    "checkpoint_every": 1000, "log_every": 100,
    "use_curriculum": false
  },
  "evaluate": {
    "eval_episodes": 500, "max_actions": 100, "eval_epsilon": 0.0
  }
}
```

The corresponding `BASELINE.md` pins this as the benchmark preset and documents the exact train/evaluate invocations.

10.2 Reported result and current Myriad archive state

`README.md` reports the following result on this preset, now confirmed in the Myriad archive for the single-seed progressive $n = 5$ run:

n	Budget	Success rate	Lab status
5	10,000	90.4%	archived and confirmed
5	20,000	–	started, checkpointed at 11,000, no archived eval
6	4,000	0.0%	archived old progressive run
6	5,000+	–	later resume failed from checkpoint-width mismatch

These are the numbers cited at submission time plus the June 17, 2026 Myriad status. Larger $n = 5$ budgets and any fresh $n = 6$ claim still require clean resubmission after the cluster checkout is updated.

10.3 Myriad / qsub wrappers

`scripts/myriad_stage2_lq_seed12345_progressive.sh`, `..._n5...`, `..._n6...intermediate.sh`, `..._n6...progressive` and `qsub_*` variants are minor wrappers around `run_stage2_study.sh` that set module loads, environment activation, and SGE qsub flags before delegating.

11 Output Layout

Per training run:

```
qamel/outputs/runs/<run_name>/
  config.json
  model.pt                    # final model + full training state
```

```
  train.log
  metrics.csv                  # per-window training metrics
  checkpoints/
    latest.pt                  # the most recent checkpoint
    episode_000NNN.pt          # periodic checkpoints
    best_eval.pt               # best-eval policy snapshot
    best_eval_metrics.json
  evaluations/
    eval_episodes<E>_max<M>_eps<e>_seed<s>.csv
    eval_episodes<E>_max<M>_eps<e>_seed<s>_summary.json
    eval_episodes<E>_max<M>_eps<e>_seed<s>_ent_counts.txt
    eval_episodes<E>_max<M>_eps<e>_seed<s>_swap_counts.txt
```

Shared caches:

```
qamel/outputs/logs/states/<n>_nodes.npy
qamel/outputs/logs/actions/<n>_nodes.npy
```

Legacy locations (the codebase still writes to these for notebook compatibility):

```
qamel/outputs/models/dqn_n<n>_pgen<p>_pswap<q>_<tag>.pt
qamel/outputs/results/eval_n<n>_pgen<p>_pswap<q>_<tag>.csv
qamel/outputs/results/ent_counts/<n>_nodes.txt
qamel/outputs/results/swap_counts/<n>_nodes.txt
qamel/q_table_storage/<n>_nodes.txt
```

Study archives:

```
qamel/outputs/studies/<RUN_TAG>/
  budget_<B>/
    model.pt, latest.pt, metrics.csv, train.log, config.json
    eval.csv, eval_summary.json
    best_eval.pt, best_eval_metrics.json   # if best-eval was enabled
  cross_budget_summary.csv
```

12 Known Issues and Modeling Caveats

This list is distilled from `REPORT.md` and from the code itself. Items 1–3 are scientific in nature and require physics judgment to resolve; items 4+ are implementation defects.

1. **Scope mismatch.** The RL code models a 1D chain producing a bipartite end-to-end Bell pair, while the README, the analytical files, and the OFC paper describe a 2D grid multipartite GHZ distribution with fusion. Until this is reconciled, any physics-aware contribution should declare which problem it is targeting.

2. **No fidelity / decoherence / memory lifetime.** The environment is purely combinatorial. A realistic physics extension would add per-edge fidelity, a decay law $F(t)$ between attempts, and a memory-cutoff truncation rule.

3. **Latency is not part of the MDP.** `step()` carries no physical time. The latency numbers in the notebooks are post-processed from counter maxima.

4. **Tabular exploration uses `torch.randn`.** (`qamel/agent.py:52`)

5. **Tabular terminal Q update adds Q instead of subtracting it.** (`qamel/agent.py:66`)

6. **Tabular saves are keyed only by n.** `qamel/q_table_storage/{n}_nodes.txt` silently shares storage between different $(p_{\text{gen}}, p_{\text{swap}})$ runs.

7. **Tabular train path can raise `NameError`.** `train_q_agent` in `scripts/train_qamel.py` calls `reward_shape` without importing it. The DQN path is unaffected.

8. **Legacy replay migration cannot recover timeout semantics.** The current DQN replay stores `terminated` and `truncated` separately and only masks Bellman bootstrapping on `terminated`. Legacy five-field replay entries are migrated as `terminated=done, truncated=False`; an old timeout transition therefore remains terminal after resume because the old payload did not preserve enough information to reconstruct the split.

9. **Bellman targets are validity-masked in this build.** The current code constructs `next_valid_mask` and applies it. The unmasked max is computed only for the `--debug_target_mask` diagnostic. So this issue is resolved; if it regresses, watch `_build_valid_action_mask`.

10. **Training vs. evaluation reward divergence.** Only an issue when `swap_ready_bonus` is wired into `compute_reward`. In the current build `reward_mode="base"` ignores the bonus everywhere, so the two are aligned.

11. **Success vs. reward inconsistency.** A state can be simultaneously final and bad. `get_episode_status` marks it as not-final-but-bad, which is the right thing. Verify that downstream metric code uses `get_episode_status` and not raw `check_if_final_state`.

12. **Adjacency-only observation is non-Markov w.r.t. counters.** The tabular learner and the `baseline` obs mode cannot see counter channels even though the (former) reward depends on `torch.amax` over them. With the current $\pm 100/-1$ reward this is moot, but if you reinstate a counter-dependent terminal reward, the baseline observation must be re-expanded.

13. **Existing links can be "refreshed."** `is_action_valid_given_state` counts only newly added edges in degree budgeting, but `step()` clears and re-samples acted-on edges regardless of whether they were already present. Whether this is a feature (lets the agent reroll a partially built chain) or a bug is a design call.

14. **`step()` has an inconsistent return type.** The early-exit branch on `len(connected_nodes) > 2` returns `(-100, state_copy)` instead of a state tensor. Callers don't currently hit this branch because `is_action_valid_given_state` masks it out, but the code path is a latent footgun.

15. **CLI for `counter_exposed_plus_ready`.** The current CLI handles this mode correctly end to end (see `DQN_OBS_MODES` in both scripts). Keep this mode covered by smoke tests so it does not regress.

16. **Myriad checkout drift can invalidate submitted jobs.** The June 17 Stage-0 escalation array failed before training because Myriad still had an older `scripts/train_qamel.py` that rejected the newer `--dueling`, `--double-dqn`, `--pbrs`, learning-rate, batch-size, and max-action flags. Treat cluster submissions as untrusted until `git pull --ff-only` and a Python import check have succeeded on Myriad.

17. $n = 6$ **checkpoint-width compatibility.** The current hidden-width formula gives $H = 768$ for $n = 6$ with `counter_exposed_plus_ready`. Legacy $n = 6$ checkpoints with $H = 512$ cannot be resumed into the current `DQNNet`; they should be kept as historical artifacts and future $n = 6$ runs should use a fresh run tag.

18. **Incomplete $n = 5$ 20k continuation.** The strongest archived $n = 5$ Myriad result is still the 10,000-episode evaluation at 90.4% success. A later continuation wrote `episode_011000.pt` but no `budget_20000` archive, so it should not be cited as a completed 20k result.

19. Notebooks pin Windows paths; `notebooks/agent_latencies.py` has an indentation bug at line 34 that breaks execution.

20. No tests. No pinned versions.

21. State enumeration scales as $2^{\binom{n}{2}}$ before filtering, so the tabular pathway dies at $n = 7$ in practice.

22. Action enumeration scales roughly 3^{n-2}, observed values listed in §5.

23. Environment stepping is a pure Python loop over edges and swap nodes; throughput is CPU-bound.

13 Performance Levers

For collaboration aimed at *maximising performance*, the following are the highest-leverage knobs and changes.

13.1 In-loop hyperparameters worth sweeping

- ε-decay schedule. `eps_decay_steps=10000` is tight for $n = 6$ where many more environment steps are needed.
- Learning rate. 10^{-3} with Adam is a default; Huber loss may tolerate 5×10^{-4} with better stability.
- Target update period. 1000 environment steps is conservative; on $n = 5$ with ∼13 steps per episode this is one update every ∼77 episodes.
- Replay capacity vs. training budget. 50k capacity vs. a 200k-episode budget means old transitions are evicted long before training ends; consider raising or switching to prioritised replay.
- Batch size. 64 is small for a 768-wide MLP; up to 256 is cheap.

13.2 Reward-shape candidates (require physics input)

- Reintroduce a counter-aware terminal bonus, e.g. $+R \cdot e^{-\beta T_{\mathrm{mem}}(s)}$ to model decoherence-driven preference for fewer attempts per memory slot.
- The implemented `--pbrs` option uses $\Phi(s)$ equal to the contiguous elementary-link prefix length from node 0. A richer potential such as the size of the connected component touching node 0 could be tested, provided it remains state-only and bounded.

- Penalise null actions explicitly to discourage time-padding when no useful generation/swap is available.

13.3 Action-space restructuring

- Replace the per-state validity scan (currently $O(|\mathcal{A}|)$ matrix checks per step) with a precomputed lookup keyed on the binary signature of the current $s[0]$ upper triangle.
- Factor the action into a product of per-edge generation bits $\times$ per-node swap bits and use either a factored Q-head or an autoregressive policy. This cleanly cuts the head from $|\mathcal{A}|$ to $\mathcal{O}(n)$ outputs.

13.4 Curriculum and exploration

- The default curriculum boundaries $(5000, 10000)$ relative to a 10000-episode total mean only the first two horizons are used. For $n = 6$ progressive runs (which target $\geq$ 100k episodes), `--curriculum_boundaries` should be retuned in proportion.
- Combine `--prefer_swap_when_ready_train` with a lower ε floor; the prefer-swap heuristic is already strong inductive bias.

13.5 Throughput

- Vectorise `RepeaterChain.step` over a batch of episodes; the inner loops are trivially batchable since edges and swap nodes are independent.
- Move replay tensors to GPU on insertion (currently they are CPU-resident).
- Cache the valid-action mask per unique adjacency signature; the mask is a pure function of $s[0]$ given a fixed action basis.

14 File-by-File Reference

14.1 `qamel/environment.py`

Defines `RepeaterChain(n, pgen, pswap, device)` with `reset()` returning the zero $(3, n, n)$ tensor and `step(state, action)` implementing the dynamics in §5. No internal state besides $n, p_{\text{gen}}, p_{\text{swap}}$, device — the class is stateless across episodes, with the state tensor passed explicitly.

14.2 `qamel/utils.py`

Action and state enumeration (`generate_all_actions`, `generate_all_valid_actions`, `generate_all_states`, `generate_all_valid_states`); terminal predicates (`check_if_final_state`, `check_if_bad_state`); the linear ε schedule (`linear_schedule`); the reward (`reward_shape`, `compute_reward`); PBRS potentials (`chain_progress_potential`, `chain_progress_potential_batch`) and their hand-crafted test helper; swap-ready counting (`count_swap_ready_nodes`); episode status reducer (`get_episode_status`); state-conditional action validity (`is_action_valid_given_state`); legacy ID lookups (`get_state_id`, `get_action_id`); and the post-hoc operation count reader (`get_operation_count`).

14.3 `qamel/agent.py`

Tabular Q-learning class with the bugs noted in §12 (items 4–6). Useful as a structural test for $n \leq 5$; not competitive with the DQN.

14.4 `qamel/dqn.py`

`preprocess_obs` converts a $(3, n, n)$ state tensor to the network input under each `obs_mode`; `DQNNet` and `DuelingDQNNet` are the MLP variants described in §6; `build_dqn_net` is the architecture factory used by training and evaluation.

14.5 `scripts/train_qamel.py`

Roughly 1325 lines. Structure:

- Top: `train_q_agent` (`:26`) for the tabular path; this branch calls `reward_shape` without importing it and can raise `NameError`. The DQN path is the supported one.
- Hyperparameter classes `hyperparameters` (tabular) and `dqn_hyperparameters` (DQN) near `:74--89`.
- Replay buffer with state-dict serialisation for checkpoint resume (`:105--133`).
- Path builders, JSON helpers, metrics CSV writer (`:135--218`).
- RNG capture/restore for deterministic best-eval (`:233--275`).
- Checkpoint payload construction and validation (`:277--410`).
- Action-validity helpers, including the batched `_build_valid_action_mask` used in DQN target computation (`:428--469`).
- `_evaluate_policy_snapshot` for best-eval (`:471--556`).
- `train_dqn_agent` itself (`:558--981`).
- CLI argument parser and orchestration (`:997--end`).

14.6 `scripts/evaluate_qamel.py`

Roughly 650 lines. Notable functions:

- `_filter_eval_action_indices`: applies the ablation flags (§7.2).
- `_summarize_state`, `_summarize_action`: build the per-step trace records.
- `evaluate_q_table`: the loop body; despite the name it also handles the DQN model path when `obs_mode` is a DQN mode.
- `run_evaluation`: assembles paths, dispatches, and writes summary JSON.

The script writes both legacy and run-local artifacts (see §11).

14.7 scripts/head_to_head.py

Standalone seeded comparison harness for `dqn_greedy`, `dqn_swapprefer`, and `heuristic`. It reads a run directory, loads `checkpoints/best_eval.pt` when present (falling back to `model.pt`), reconstructs the network from `net_arch`, evaluates each policy/seed pair, writes the requested JSON schema, and prints a Markdown summary table.

14.8 scripts/run_stage2_study.sh

Bash driver for the budget sweep (§8). Reads ∼30 env vars, archives artifacts per budget, rebuilds the cross-budget summary CSV via an embedded Python heredoc.

14.9 Myriad wrappers

`scripts/myriad_stage2_lq_seed12345_progressive.sh`, `scripts/myriad_stage2_lq_n5_seed12345_progressive.sh`, `scripts/myriad_stage2_lq_n6_seed12345_progressive.sh`, `scripts/myriad_stage2_lq_n6_seed12345_intermediate.sh` `scripts/myriad_stage2_swapaware_seed12345.sh`, and their `qsub_*` counterparts: set cluster-specific module loads, Python virtualenv activation, and SGE qsub directives, then export the appropriate env vars and `exec run_stage2_study.sh`.

14.10 scripts/local_pilot_stage2_lq_seed12345_progressive.sh

Local laptop equivalent with smaller budgets (50/200/500/1000) for workflow validation only. Reported to take ∼14.5 minutes for 50 episodes on CPU.

14.11 analytical_solution/analytical_equations.py

Markov / Shchukin / Bernardes / fusion-augmented closed-form expectations (§9).

14.12 analytical_solution/monte_carlo.py

`MonteCarloFusion`: recursive sampler for the nested-swap waiting-time distribution, with optional fusion sampling. Top-level helper `monte_carlo(vertices, swap_samples, fusion_samples, p, q, k, ind=False)` returns either just the swap-side waiting-time samples or the full tuple of (time, t_p, t_q, t_k) samples.

14.13 analytical_solution/utils.py

Manhattan distance for the grid topology, power-of-2 decomposition (used to choose nesting depth in the Monte-Carlo sampler), batched-protocol runner `monte_carlo_sim`, and `normalize_array`.

14.14 Notebooks

- `notebooks/verify_simulations.ipynb`: cross-checks Monte-Carlo histograms against `bernardes_eq`, `markov_approach`, `including_fusion`. Supports the OFC paper figures.
- `notebooks/calculate_ratio_and_latencies.ipynb`: post-processes operation-count statistics into classical / quantum latency contributions using fixed operation times and fibre propagation $\tau = L/c_{\text{fibre}}$.

- `notebooks/analysis_of_results.ipynb`: the working analysis notebook for the DQN study, including the README's $n = 5$ success-rate table.
- `notebooks/stage_a_diagnostics_checkpoint_sweep.ipynb`: per-checkpoint diagnostics from a stage-2 run (return curves, action distributions, swap-readiness fraction, ablation deltas).
- `notebooks/paper_figures.ipynb`: builds publication-style figures from the current evaluation outputs. Generated image artifacts live under `figures/`.
- `notebooks/agent_latencies.py`: intended helper to drive `run_evaluation` for several n values and write a combined results file. Currently broken (indentation bug at line 34; also assumes the legacy results-directory layout).

14.15 Top-level documents

- `README.md`: project framing and quick-start.
- `BASELINE.md`: canonical phase-2 preset; train and evaluate commands.
- `MYRIAD_STAGE2.md`: cluster workflow and overwrite semantics.
- `MYRIAD_WORKFLOW.md`: current Myriad login, module, venv, git, qsub, monitoring, and known-failure checklist.
- `REPORT.md`: a long, audit-style technical report. The single best document for an external collaborator who wants to understand what was checked in at a given snapshot. Many findings in REPORT.md are now resolved; the list in §12 above is the up-to-date subset.
- `OFC_paper_2025.pdf`: published framing; broader scope than the executable RL.
- `LICENSE`: MIT.

15 Glossary

Elementary link
An entangled Bell pair shared between two nearest-neighbour nodes on the chain. Modelled here as a binary present/absent indicator.

Generation attempt
A Bernoulli trial that creates an elementary link with probability p_{gen}. Increments the generation counter on both endpoint nodes whether it succeeds or fails.

Swap attempt
At an interior node k with two incident links to neighbours j, ℓ, a Bernoulli trial with success probability p_{swap} that creates a direct link (j, ℓ) and consumes the two incident links.

Bad state
Any state in which some endpoint has degree > 1 or some interior node has degree > 2. Treated as a failure terminal.

Final state
$s[0, n-1] \neq 0$, i.e. a direct entangled link between the two endpoints exists.

Swap-ready node
An interior node whose current degree is exactly 2 and therefore can usefully attempt a swap.

Counter channel
The $s[1]$ or $s[2]$ slice of the state tensor, recording per-node generation or swap attempts on the diagonal.

Curriculum phase
A range of episode indices during which the per-episode action horizon is held below `max_actions`, used to ease early training.

Best-eval checkpoint
The training-time snapshot whose offline rollout maximised (success rate, mean return, low mean steps), in lexicographic order.

Budget
In §8, the cumulative number of training episodes at which a checkpoint+evaluation pair is archived.

16 What to do first as the collaborating Claude

Compressed advice for the first message of the next physics-aware Claude reading this document:

1. Confirm or refute the scope mismatch in §3: do we want to keep this as a 1D chain Bell-pair scheduler, or extend the RL environment to match the OFC paper's grid+fusion scope?
2. If chain: propose a fidelity / memory-lifetime extension to `RepeaterChain` that is compatible with the existing 3-channel state tensor (e.g. replace binary $s[0]$ entries with a Werner parameter, add a $s[3]$ channel for memory age). State the reward modification this requires.
3. If grid+fusion: propose a tensor state representation, an action factorisation, and how to plug `generate_tpm` into the environment's transition (or whether to replace it entirely).
4. Independently, fix items 4–6, 13 in §12: these are clean, mechanical fixes that don't require a physics decision and unblock the tabular learner as a regression sentinel.
5. Use `configs/phase2_baseline_n5_plus_ready.json` as the regression preset for any proposed change. Use `BASELINE.md`'s smoke-check commands to verify in $\sim$2 episodes before kicking a real budget.